

Constraint Satisfaction Problem I

CSE 4617: Artificial Intelligence



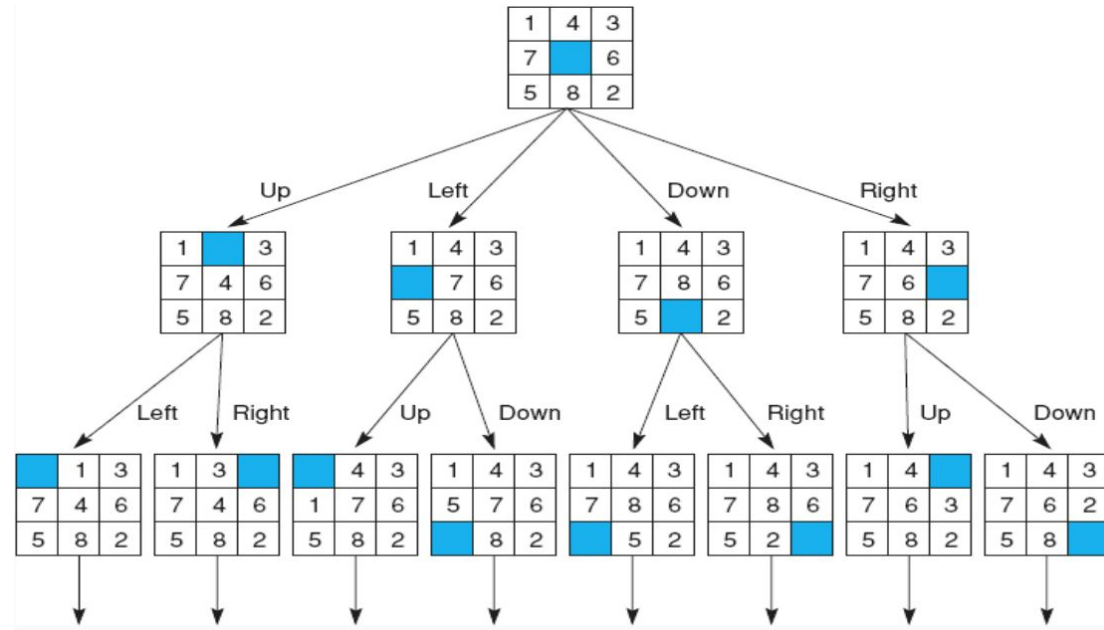
Isham Tashdeed
Lecturer, CSE



When can search be used?

Assumptions:

- Single agent
- All the states can be fully observed
- States are discrete
- Actions taken by the agent are deterministic



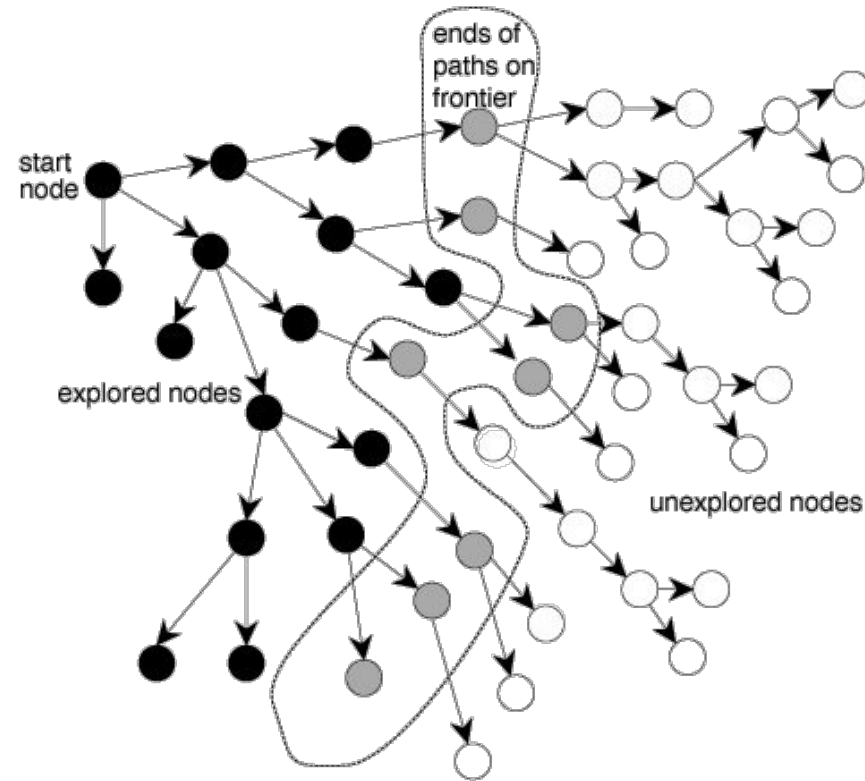
When can search be used?

Planning:

- The path leading to the goal is as important as reaching the goal
- Paths have various cost/depth
- Heuristics are a helping hand

Identification:

- Reaching the goal is important, not the path
- All paths are the same depth (How?)
- CSPs are special identification problem

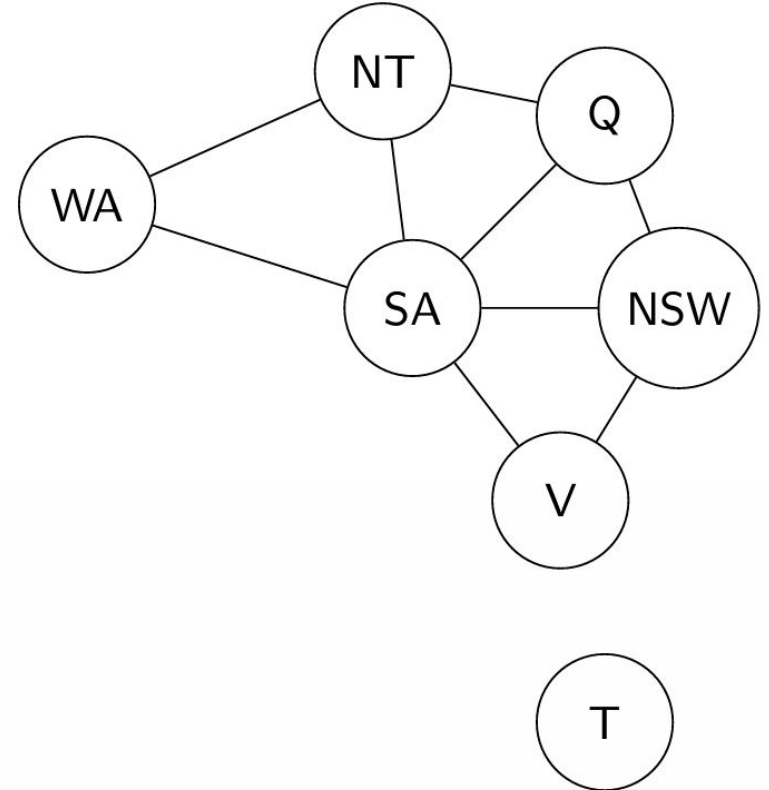
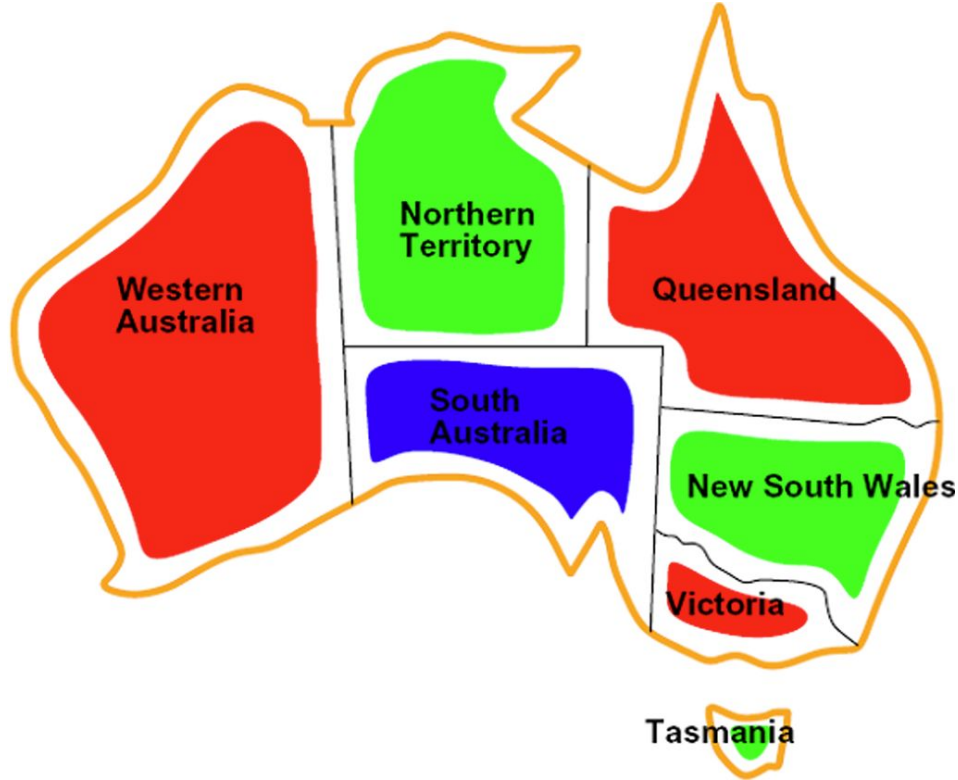


Constraint Satisfaction Problem

Examples of CSPs

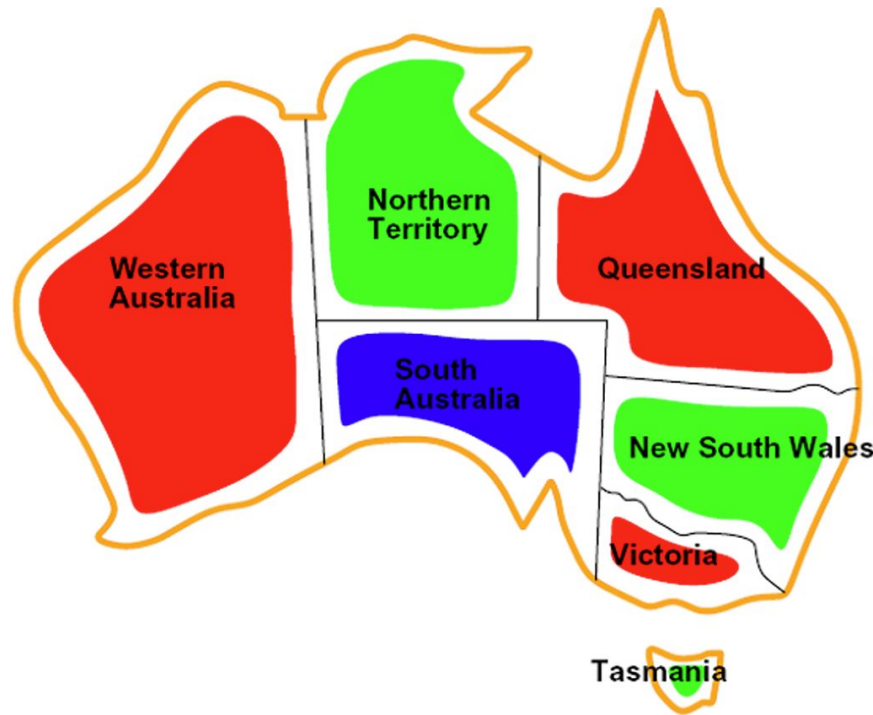


Examples of CSPs



Examples of CSPs

- Variables
 - WA, NT, Q, NSW, V, SA, T
- Domains
 - $D = \{\text{red, green, blue}\}$
- Constraints → Adjacent regions must be different colored
 - Implicit: $WA \neq NT, \dots$
 - Explicit: $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), (\text{blue, green}), \dots\}$
- Solution
 - An assignment of variables that satisfy all constraints

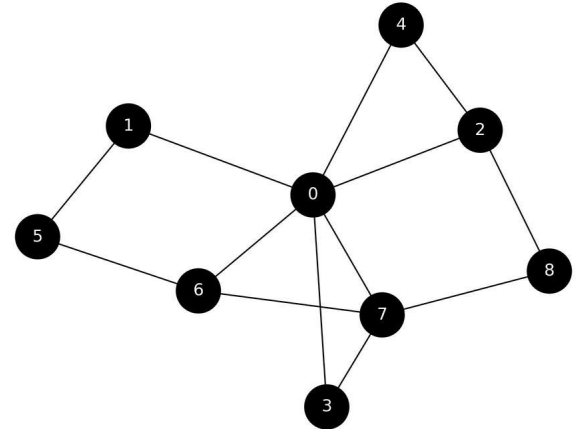


Examples of CSPs

CSPs are a special subset of search problems:

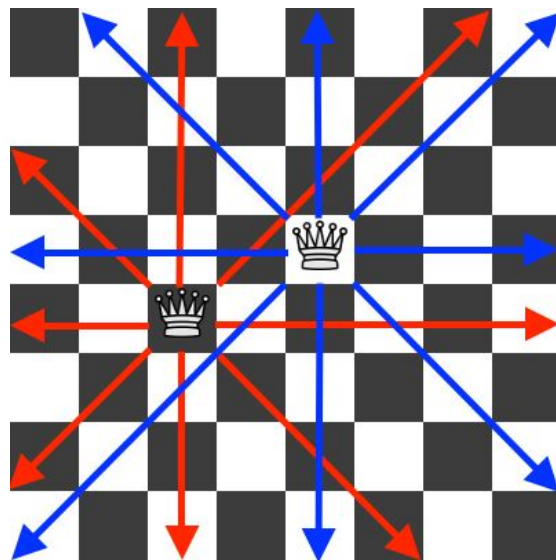
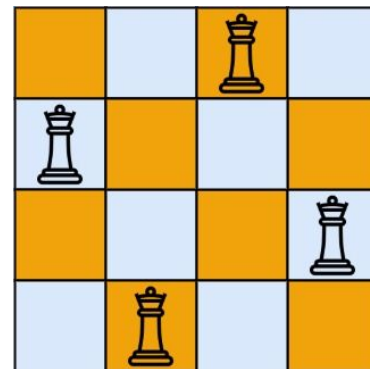
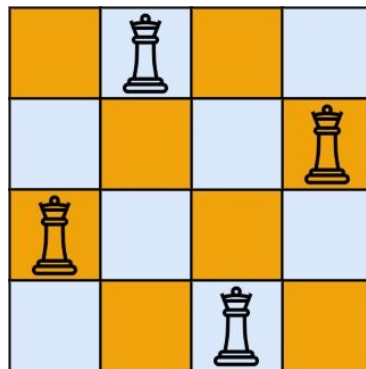
- A state is defined by variable X_i with values from a domain D
- Goal test is a **set of constraints** denoting valid combinations of values assigned to X_i

Why is it different from regular search problems?



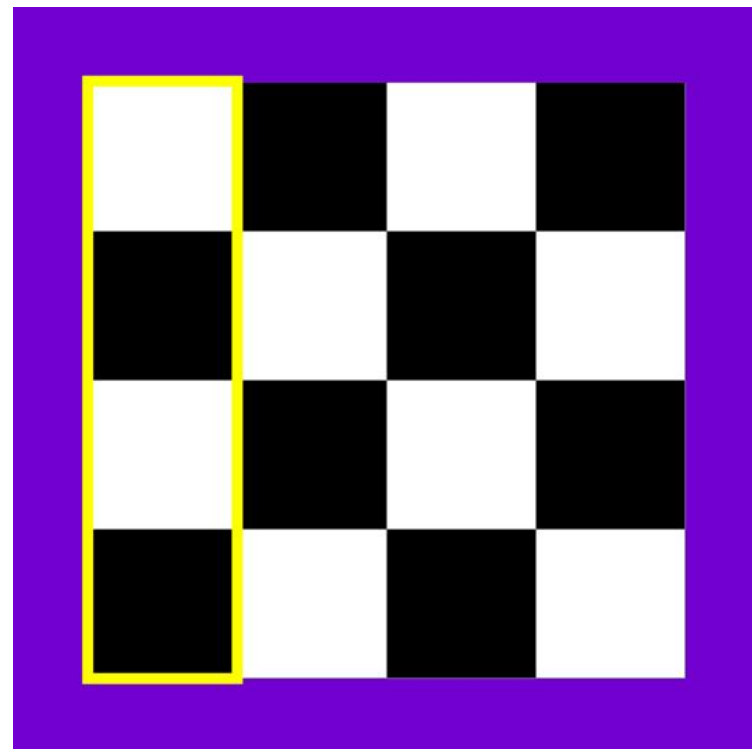
Examples of CSPs

- Variables
 - X_{ij}
 - Total $n \times n$ variables
- Domains
 - $D = \{0, 1\}$
- Constraints → No queen should attack the others
 - Explicit Constraint?
 - What if all variables are 0?
- Solution
 - An assignment of variables that satisfy all constraints



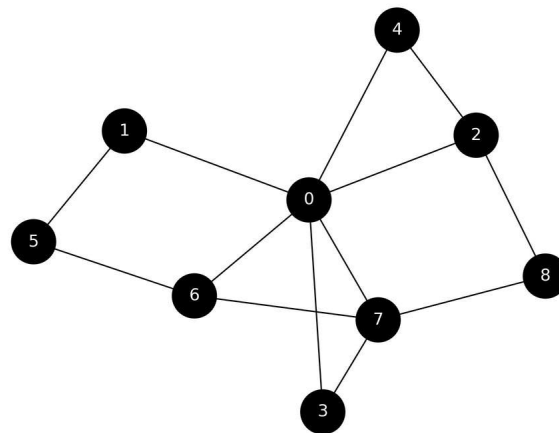
Examples of CSPs

- Variables
 - Q_k
 - Total $n \times n$ variables
- Domains
 - $D = \{1, 2, 3, \dots, N\}$
- Constraints $\rightarrow$ No queen should attack the others
 - Explicit Constraint?
 - $(Q_i, Q_j) \in \{(1, 3), (1, 4), \dots\}$ where $|i - j| = 1$
- Solution
 - An assignment of variables that satisfy all constraints



Constraint Graphs

- Binary CSPs $\rightarrow$ Each constraint relates to two variables at most
- Binary constraint graph $\rightarrow$ Nodes are variables, arcs define the constraints
- Use the graph structure to speed up the search algorithm $\rightarrow$ But how?



Examples of CSPs

- Variables
 - Each empty square
- Domains
 - $D = \{1, 2, 3, \dots, 9\}$
- Constraints → No queen should attack the others
 - Unary constraint for given values
 - 9-way all-diff for each column
 - 9-way all-diff for each row
 - 9-way all-diff for each region
 - No empty squares left
- Solution
 - An assignment of variables that satisfy all constraints

		2		9	1		3	
			2		6	1		7
	7			3		9		
7		3		6		8		
	6	5	1		7	2	9	
		9		2		7		4
		1		7			8	
2		4	6		8			
	8		9	5		3		

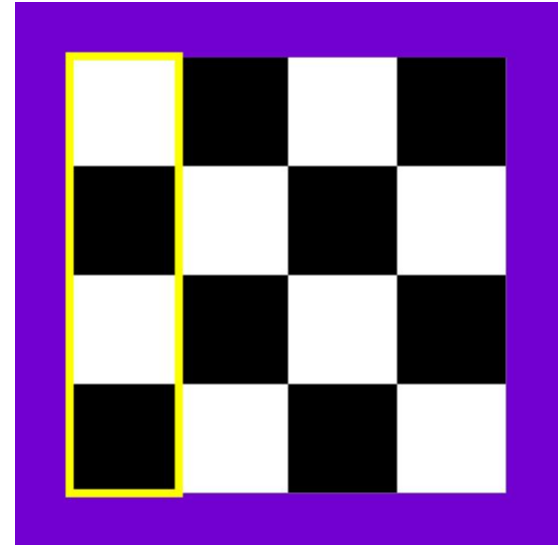
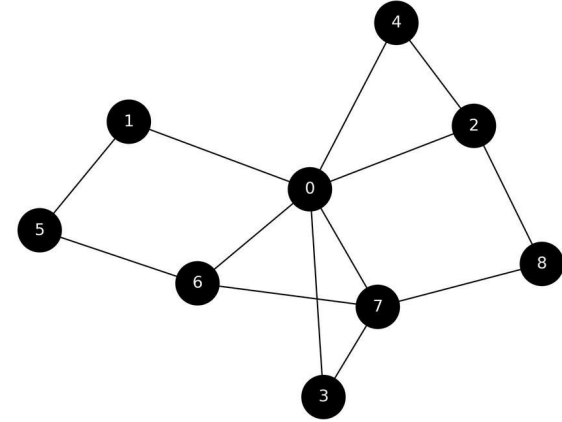
Time: 1:31

Types of Constraints

- Unary constraints involve a single variable
 - $SA \neq \text{green}$
- Binary constraints involve pairs of variables
 - $SA \neq WA$
- Higher-order constraints involve more than 2 variables
 - Sudoku column constraint
 - Sudoku row constraint
- Preferences
 - Often represented as cost of variable assignment $\rightarrow$ Preferred values have low cost
 - Optimization problem on top of a CSP

Solving CSPs

- State $\rightarrow$ The partial assignment of values
- Start State $\rightarrow$ Empty assignment $\rightarrow \{\}$
- Successor Function $\rightarrow$ Assigns a value to an unassigned variable
- Goal test $\rightarrow$ All the variables have values assigned to them & No constraint is violated

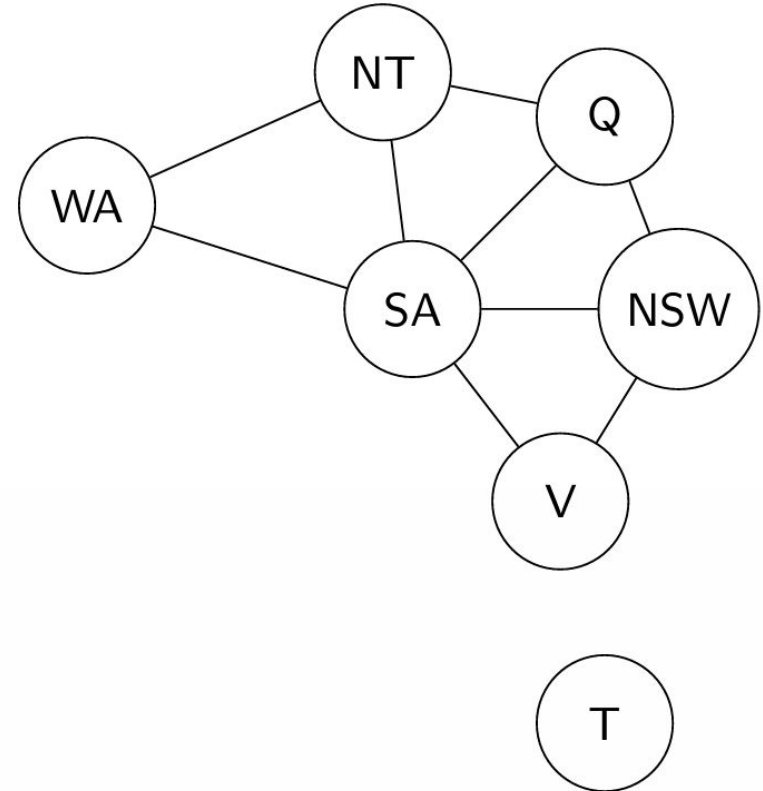


Solving CSPs

- State $\rightarrow$ The partial assignment of values
- Start State $\rightarrow$ Empty assignment $\rightarrow \{\}$
- Successor Function $\rightarrow$ Assigns a value to an unassigned variable
- Goal test $\rightarrow$ All the variables have values assigned to them & No constraint is violated

What happens if we use BFS?

What happens if we use DFS? $\rightarrow$ Naive DFS

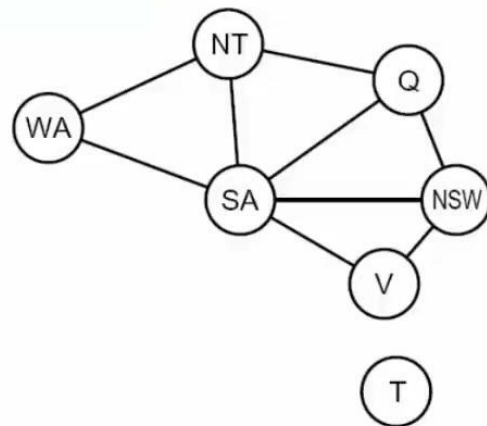


Naive DFS

Search Methods

- What would BFS do?

- What would DFS do?



[demo: dfs]

Backtracking Search

Backtracking Search

- One variable at a time
 - Only need to consider assignments to a single variable at each step
 - Variable assignments are commutative → Any ordering is OK!
 - [WA = red then NT = green] same as [NT = green then WA = red]
- Check constraints as you go
 - Consider only values which do not conflict with previous assignments
 - “Incremental goal test”
- DFS + these two improvements = Backtracking Search

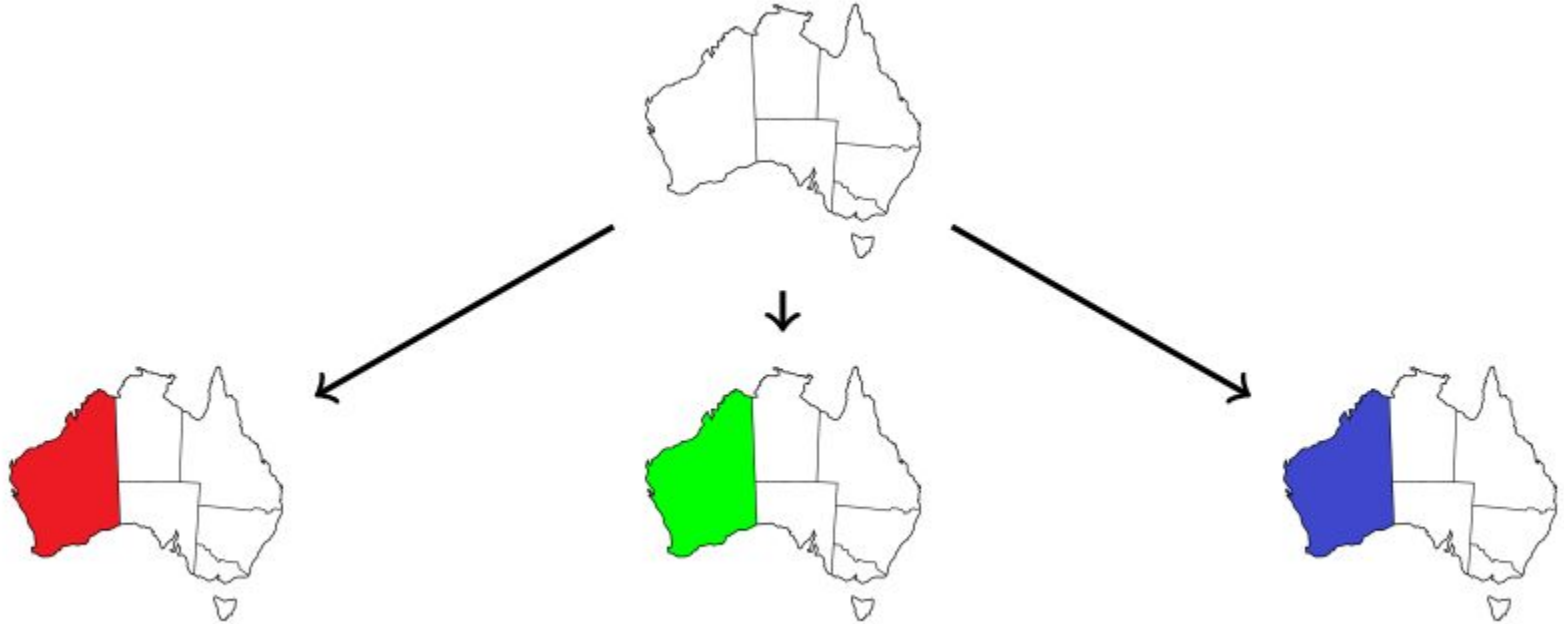
Backtracking Search

The screenshot shows a web browser window with the address bar displaying `beta.cs188.org/exercises/csps/forward_checking/forward_checking.html`. The main content area features a 3x3 grid of circular nodes connected by horizontal, vertical, and diagonal lines. To the right of the grid is a control panel with the following sections:

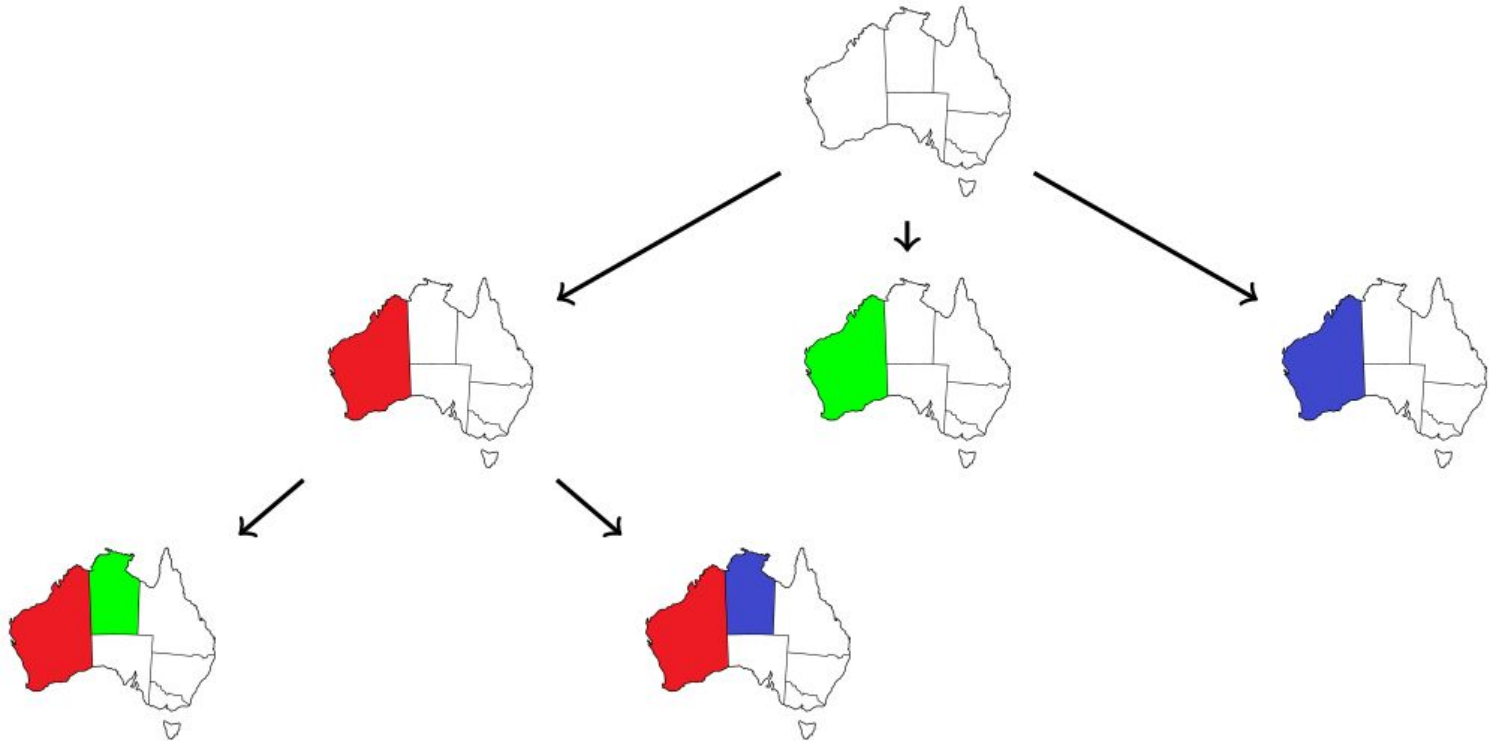
- Graph**: A dropdown menu set to "Simple".
- Algorithm**: A dropdown menu set to "Backtracking".
- Ordering**: Radio buttons for "None", "MRV", and "MRV with LCV".
- Filtering**: Radio buttons for "None", "Forward Checking", and "Arc Consistency".
- Speed**: Two input fields, "Speedup" (set to 1) and "Frame Delay" (set to 700).

Below the grid are five buttons: "Reset", "Prev", "Pause", "Next", and "Play". Below the speed controls are two more buttons: "Faster" and "x". The Windows taskbar at the bottom shows various application icons and a system tray with a 100% battery indicator, network status, and the date/time (11:46 AM, 9/4/2012).

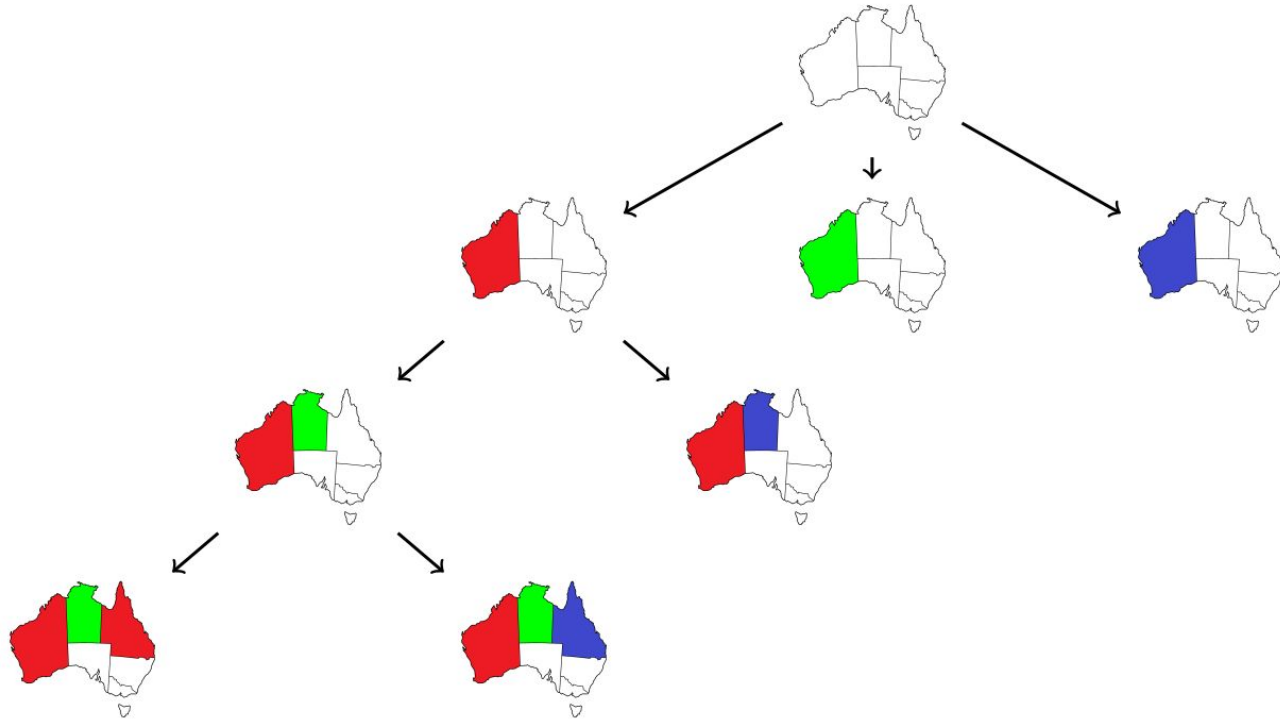
Backtracking Search



Backtracking Search



Backtracking Search



Backtracking Search

BACKTRACKING-SEARCH (*csp*) → returns a solution or failure
return RECURSIVE-BACKTRACKING ({}, *csp*)

RECURSIVE-BACKTRACKING (*assignment*, *csp*) → returns a solution or failure
if *assignment* is complete then return *assignment*
var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[*csp*], *assignment*, *csp*)
for *value* in ORDER-DOMAIN-VALUE(*var*, *assignment*, *csp*):
 if *value* is consistent with *assignment* given CONSTRAINTS[*csp*]:
 then add {*var* = *value*} to *assignment*
 result ← RECURSIVE-BACKTRACKING (*assignment*, *csp*)
 if *result* is not failure then return *result*
 remove {*var* = *value*} from *assignment*
return failure

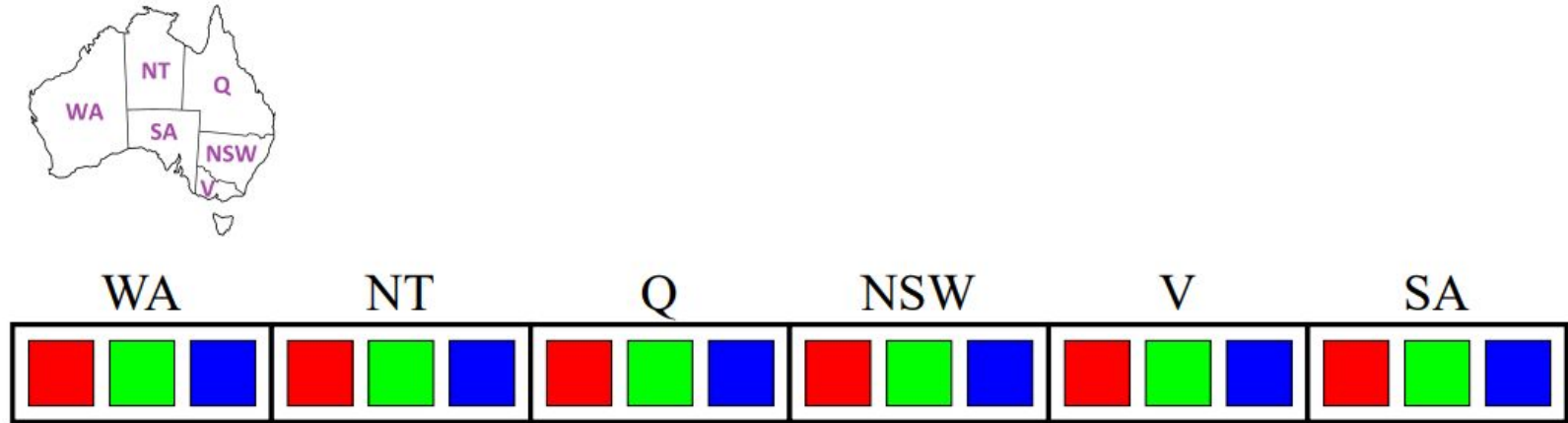
How to Improve Backtracking Search

- Filtering
 - Limiting our choices for variable assignment
 - Detects an inevitable failure early
- Ordering
 - In what order should we assign variables
 - Does it actually matter?
- Structure
 - Can we exploit the structure of a problem?

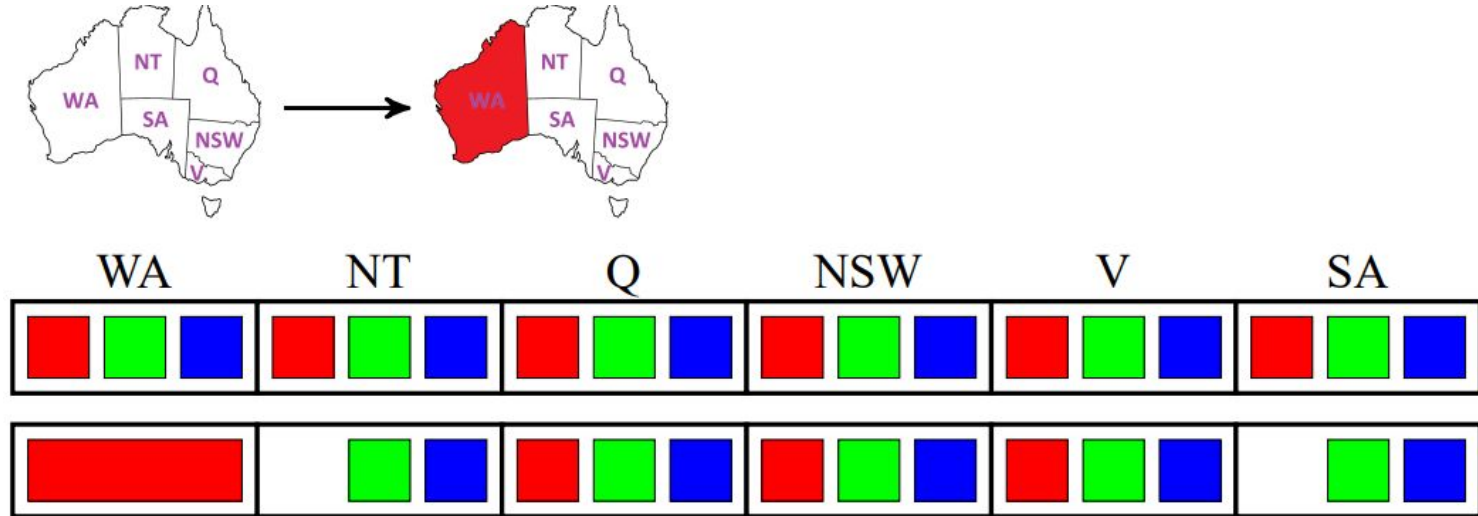
Filtering: Forward Checking

Filtering: Forward Checking

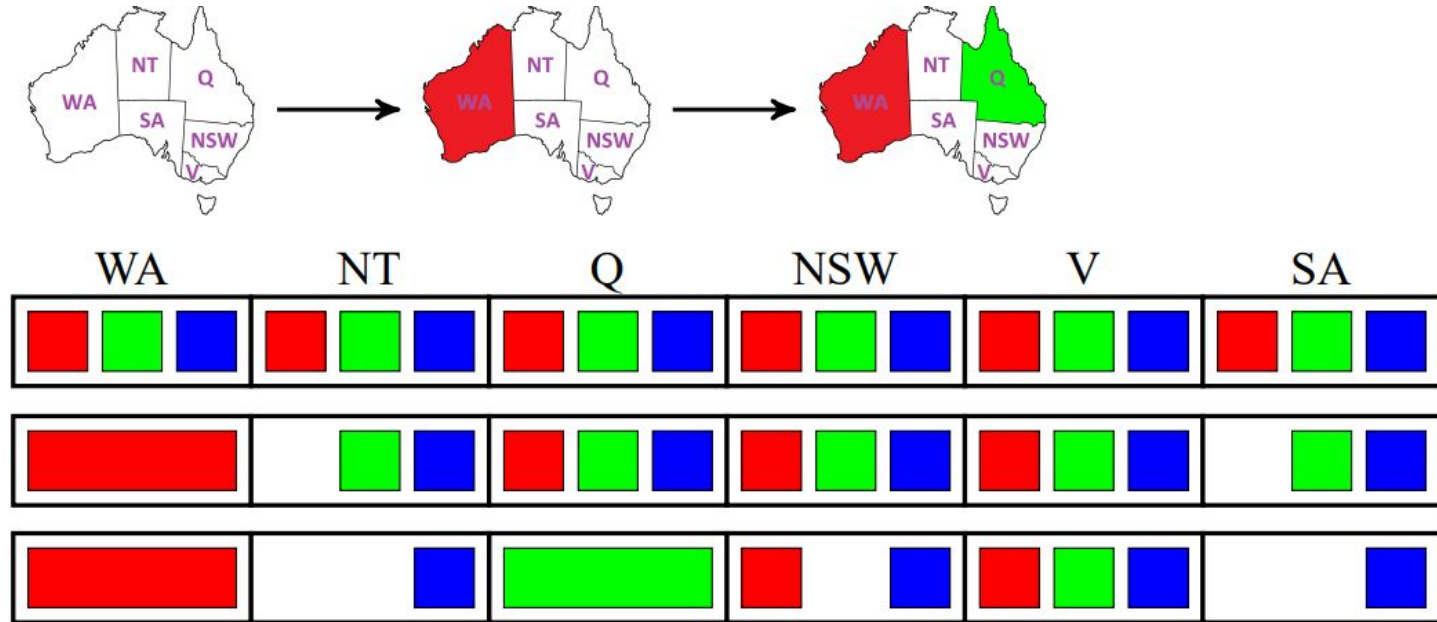
- Keep track of the domains for all unassigned variables
- Remove values that violate a constraint when added



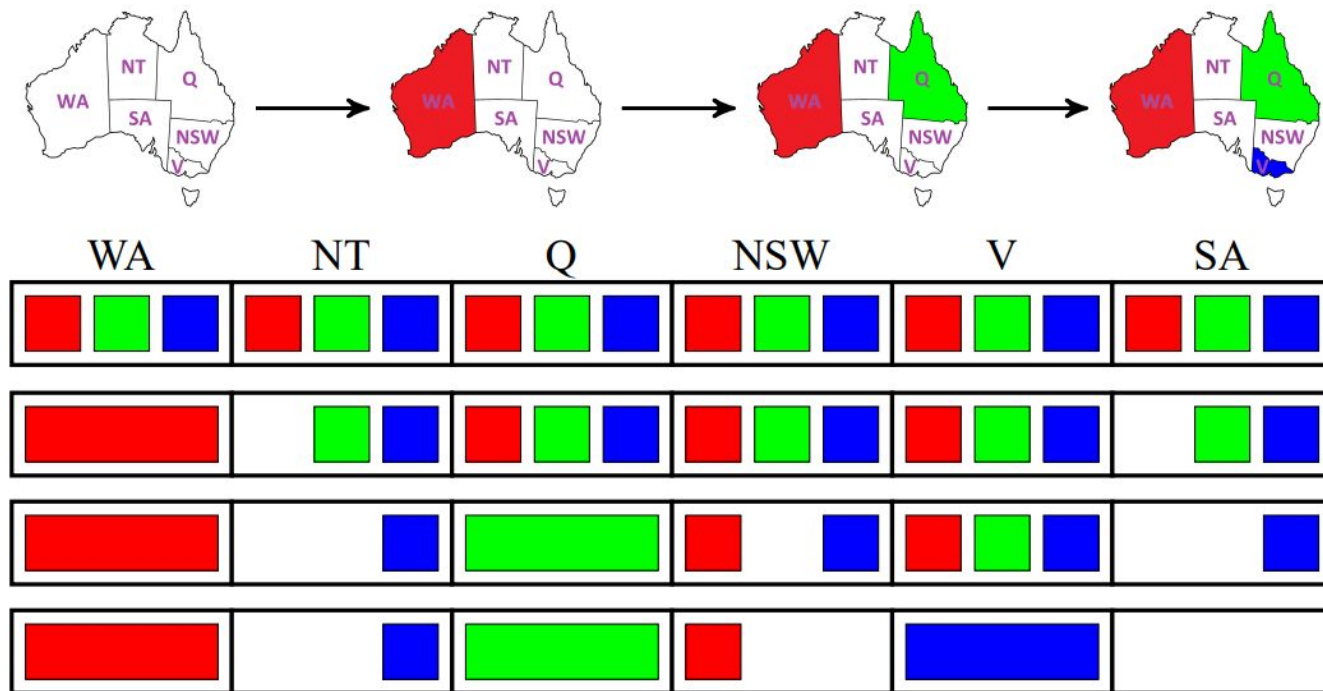
Filtering: Forward Checking



Filtering: Forward Checking



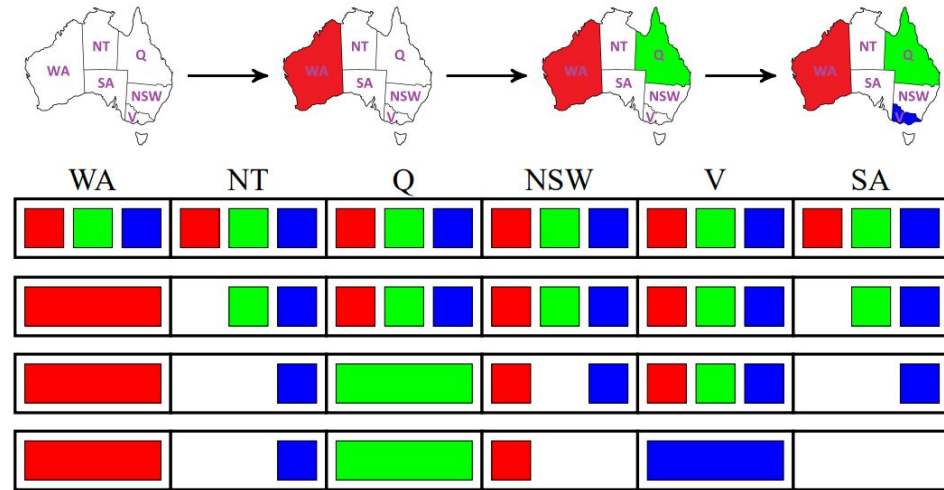
Filtering: Forward Checking



Filtering: Forward Checking

- Propagates information from assigned to unassigned variables
- Does not check pairs of unassigned variables for early detection of failures

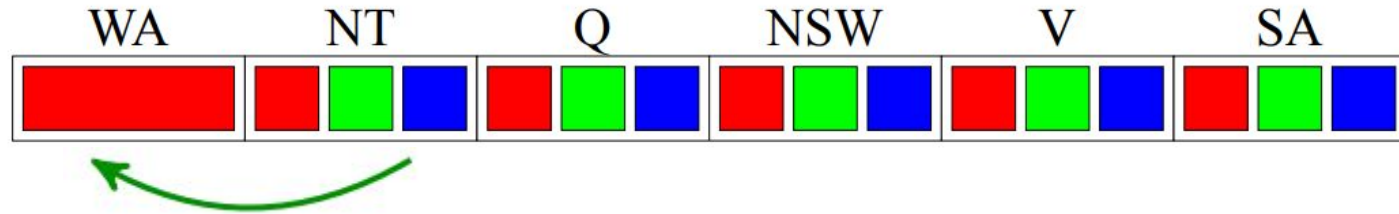
We need to propagate the changed constraints from variable to variable



Filtering: Arc Consistency

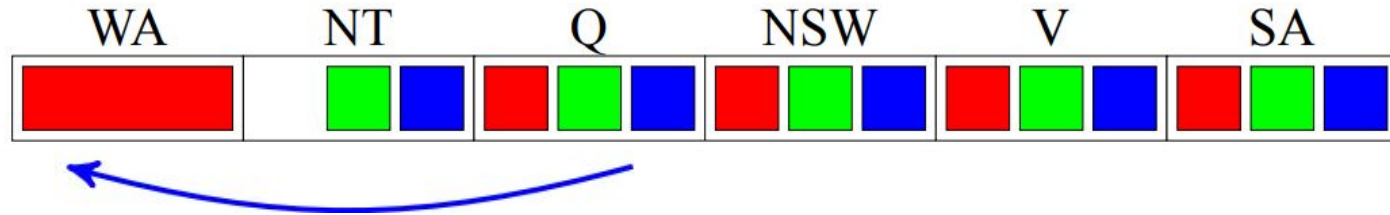
Filtering: Forward Checking

An arc, $X \rightarrow Y$, is consistent if and only if for every x in the tail, there is some y in the head which could be assigned without violating a constraint



Filtering: Forward Checking

An arc, $X \rightarrow Y$, is consistent if and only if for every x in the tail, there is some y in the head which could be assigned without violating a constraint



Delete from the tail!

Forward checking: Enforcing consistency of arcs pointing to each new assignment

Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



If X loses a value, arcs leading to X need to be rechecked!

Filtering: Arc Consistency

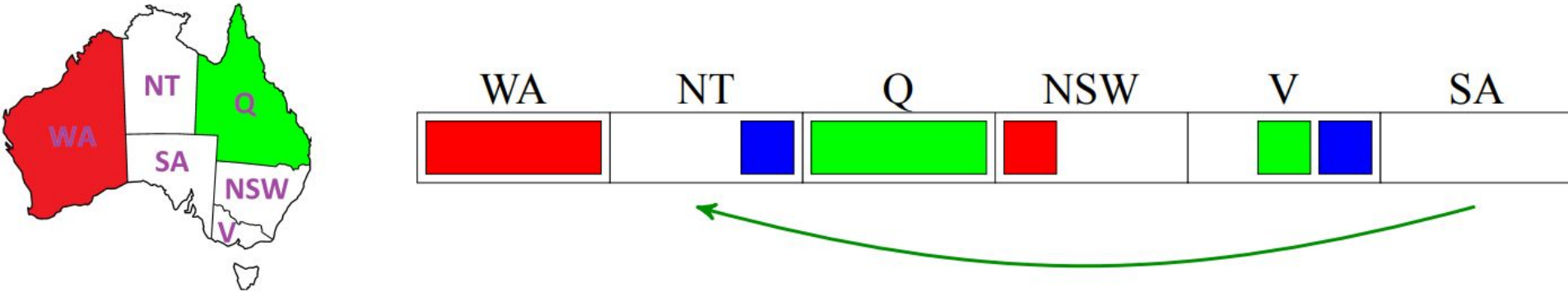
A simple form of propagation makes sure **all arcs** are consistent



If X loses a value, arcs leading to X need to be rechecked!

Filtering: Arc Consistency

A simple form of propagation makes sure **all arcs** are consistent



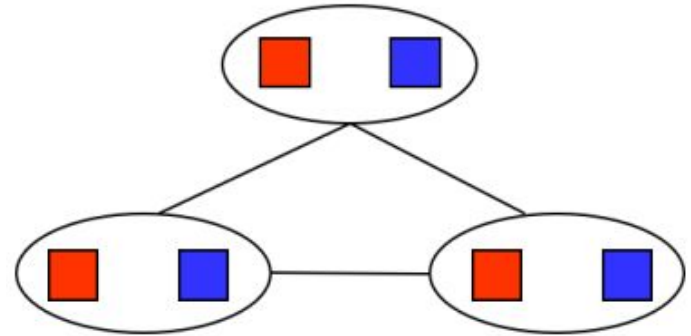
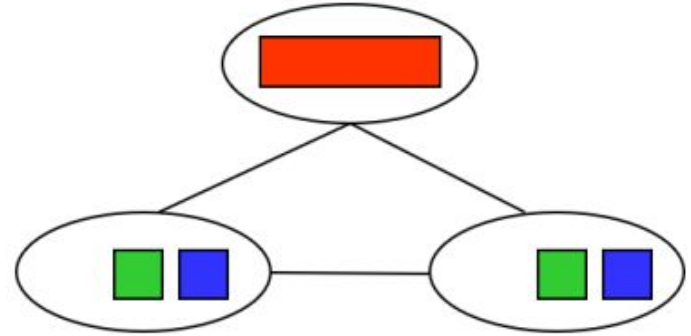
- Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment → Complexity?

Filtering: Arc Consistency

After enforcing arc consistency:

- Can have one solution left
- Can have multiple solutions left
- Can have no solutions left (and not know it)

Arc consistency only checks pairs :(

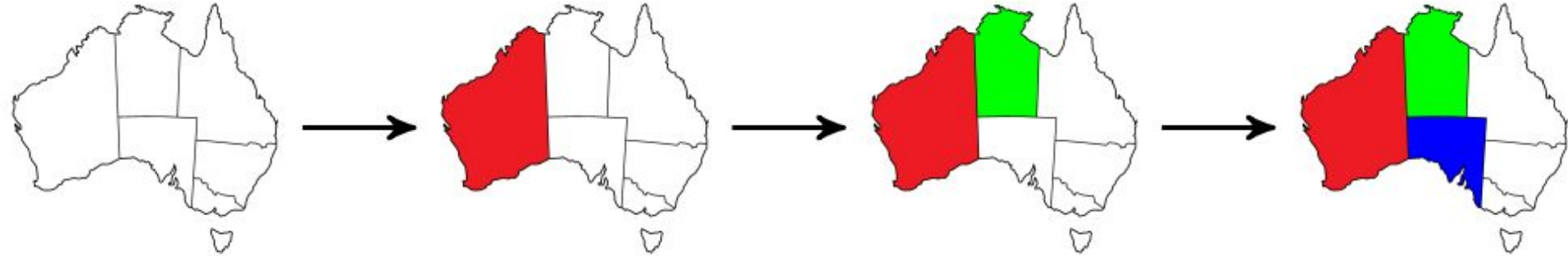


Ordering

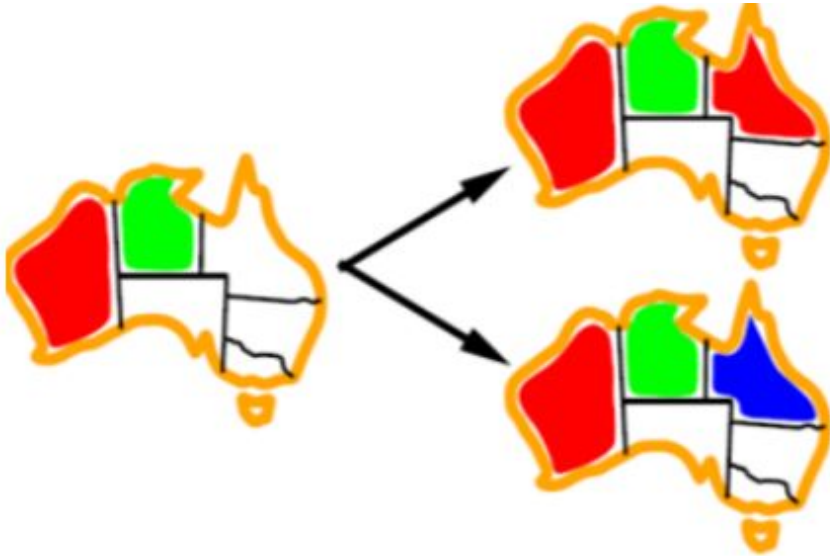
Ordering: Minimum Remaining Values

Choose the variable with the fewest legal left values in its domain

- Why min rather than max? → Fails faster
- This is also called the most constrained variable



Ordering: Least Constrained Value



- Given a choice of variable, choose the least constraining value
- The one that rules out the fewest values in the remaining variables
- Must do further filtering to determine which value is the least constrained
- Why least rather than most?
 - Leave more options for others

Additional Resources

- [Backtracking Search Simulator](#)
- [Procedural Generation using Constraint Satisfaction](#)

Thank you